

Proyecto Final — Módulo de Categorías

Objetivo

Extender el sistema de tareas (Todo) con un módulo de **Categorías** que permita organizar las tareas. El estudiante deberá implementar el backend completo, completar endpoints faltantes de Users, documentar todo en Swagger, y opcionalmente extender el frontend y desplegar a un servicio cloud.

Requerimientos Obligatorios

1. Modelo de datos — Categorías

Agregar al schema de Prisma (`packages/backend/prisma/schema.prisma`) una entidad `Category`:

```
model Category {
  id      String @id @default(uuid())
  name    String @unique
  color   String?
  userId  String
  user    User   @relation(fields: [userId], references: [id])
  todos   Todo[]
}
```

Actualizar el modelo `User` para incluir la relación:

```
model User {
  id          String      @id @default(uuid())
  email       String      @unique
  name        String?
  password    String
  role        Role        @default(CLIENT)
  status      UserStatus  @default(ACTIVE)
  todos       Todo[]
  categories  Category[]
}
```

Actualizar el modelo `Todo` para incluir la relación con `Category` (opcional):

```
model Todo {
  id          String      @id @default(uuid())
  title       String
```

```

description String?
completed Boolean @default(false)
dueDate DateTime?
userId String
user User @relation(fields: [userId], references: [id])
categoryId String?
category Category? @relation(fields: [categoryId], references: [id])
}

```

Nota: Después de modificar el schema, ejecutar `npm run prisma migrate dev` para generar la migración.

2. Backend — Módulo Categories

Crear el módulo completo siguiendo la arquitectura existente en `packages/backend/src/contexts/tasks/todo/`. La estructura debe ser:

```

packages/backend/src/contexts/tasks/category/
├── application/
│   ├── dto/
│   │   ├── create-category.dto.ts
│   │   └── update-category.dto.ts
│   └── category.service.ts
├── domain/
│   ├── category.entity.ts
│   └── category.repository.ts
├── infrastructure/
│   └── prisma-category.repository.ts
├── category.module.ts
└── category.service.spec.ts

```

Endpoints requeridos

Método	Ruta	Descripción	Swaggerdecorator requerido
GET	<code>/categories</code>	Listar categorías del usuario	<code>@ApiTags, @ApiOperation</code>
GET	<code>/categories/:id</code>	Obtener una categoría por ID	<code>@ApiParam, @ApiResponse(404)</code>
POST	<code>/categories</code>	Crear una categoría	<code>@ApiCreatedResponse</code>
PATCH	<code>/categories/:id</code>	Actualizar una categoría	<code>@ApiResponse, @ApiParam</code>
DELETE	<code>/categories/:id</code>	Eliminar una categoría	<code>@ApiResponse(200), @ApiParam</code>

DTOs

CreateCategoryDto:

- `name`: string, requerido, documentado con `@ApiProperty({ example: 'Trabajo' })`
- `color`: string, opcional, documentado con `@ApiProperty({ example: '#FF5733', required: false })`

UpdateCategoryDto:

- `name`: string, opcional
- `color`: string, opcional

Ambos DTOs deben usar `class-validator` (`@IsString`, `@IsOptional`, `@IsNotEmpty`) y `class-transformer`, igual que los DTOs existentes.

Servicio

El `CategoryService` debe seguir el mismo patrón que `TodoService`:

- Inyectar `CategoryRepository` (abstracto)
- `findAll()`: retorna todas las categorías del usuario autenticado
- `getOne(id)`: retorna una categoría o lanza `NotFoundException`
- `create(userId, dto)`: crea y retorna la categoría
- `update(id, dto)`: actualiza parcialmente
- `deleteItem(id)`: elimina (verificar que exista antes de borrar)

Repositorio

Crear `CategoryRepository` (abstracto) y `PrismaCategoryRepository` (implementación con Prisma), siguiendo el patrón de `TodoRepository` y `PrismaTodoRepository`.

Registro del módulo

Registrar `CategoryModule` en `contexts.module.ts` para que esté disponible en la aplicación.

3. Tests — Mínimo 3 tests básicos

Crear `category.service.spec.ts` con al menos 3 tests usando el patrón existente:

```
// Ejemplo de estructura esperada
describe('CategoryService', () => {
  // Test 1: should be defined
  it('should be defined', () => { ... });

  // Test 2: findAll returns categories
  it('returns all categories', async () => { ... });
});
```

```
// Test 3: getOne throws NotFoundException when not found
it('throws NotFoundException when category not found', async () => { ...
});
});
```

Patrón de testing a seguir (ver `todo.service.spec.ts`):

- Usar `Test.createTestingModule` de `@nestjs/testing`
- Mockear el repositorio con `jest.Mocked<CategoryRepository>`
- Verificar que el servicio llama al repositorio con los argumentos correctos
- Verificar que `NotFoundException` se lanza cuando el registro no existe

Ejecutar los tests con:

```
pnpm --filter todo-backend test
```

4. Endpoints faltantes de Users

El controller de Users (`packages/backend/src/apps/api/user.controller.ts`) actualmente solo tiene `GET /users` y `POST /users`. Completar con:

Método	Ruta	Descripción	Requiere admin
GET	<code>/users/:id</code>	Obtener usuario por ID	No
PATCH	<code>/users/:id</code>	Actualizar usuario	Sí
DELETE	<code>/users/:id</code>	Eliminar usuario	Sí

- `GET /users/:id`: debe retornar el usuario sin el password
- `PATCH /users/:id`: solo el propio usuario o un admin puede actualizar
- `DELETE /users/:id`: solo un admin puede eliminar

Agregar los decoradores de Swagger correspondientes (`@ApiParam`, `@ApiOperation`, `@ApiResponse`).

5. Documentación Swagger

Verificar que la documentación Swagger esté accesible en `/docs` y que todos los endpoints (Users, Todo, Categories) aparezcan correctamente:

- Cada endpoint debe tener `@ApiOperation({ summary: '...' })`
- Cada endpoint debe tener `@ApiResponse` para los casos de éxito y error
- Los DTOs deben tener `@ApiProperty` en cada campo
- La estructura de tags debe ser: `Users`, `Todo`, `Categories`

Requerimientos Puntos Extras (Bonus)

Extra 1: Frontend — Página de Categorías

Crear una nueva página `CategoriesPage.tsx` en `packages/frontend/src/pages/` que permita:

- Listar categorías del usuario
- Crear una categoría (nombre + color)
- Editar una categoría (click para editar inline, como en `TodosPage`)
- Eliminar una categoría
- Seleccionar color con un input nativo `<input type="color">`

Registrar la ruta `/categories` en `App.tsx` dentro de `ProtectedRoute`.

Puntos adicionales:

- Agregar un filtro de categorías en la página de Todos (filtrar tareas por categoría)
- Mostrar el color de la categoría como badge en cada todo

Extra 2: Despliegue a AWS

Desplegar el sistema completo a un servicio de AWS. Opciones válidas:

Servicio	Complejidad	Descripción
EC2	Baja	Docker Compose en una instancia EC2 (similar a producción local)
ECS Fargate	Media	Containers managed, requiere ECR + Task Definition
App Runner	Baja	Deploy directo desde GitHub, sin infraestructura-managed

Requisitos para el extra de despliegue:

- La API debe estar accesible públicamente (con auth JWT)
- Swagger debe funcionar en el endpoint `/docs`
- El frontend debe conectarse al backend desplegado

Criterios de evaluación

Criterio	Puntos
Schema de Prisma correcto con migración	10
Módulo Categories completo (CRUD)	30
Tests (3 mínimos, pasando)	20
Endpoints Users completados	20

Criterio	Puntos
Swagger documentado correctamente	10
Código limpio y consistente	10
Extra: Frontend de Categorías	+10
Extra: Despliegue a AWS	+5

Total obligatorio: 100 puntos

Instrucciones de entrega

1. Hacer un fork del repositorio: <https://github.com/omarmus/todo-backend>
 2. Crear una rama `feature/final-project-categories` en tu fork
 3. Commits atómicos y descriptivos
 4. Todos los tests deben pasar (`pnpm --filter todo-backend test`)
 5. El backend debe compilar sin errores (`pnpm --filter todo-backend build`)
 6. Abrir un Pull Request contra `main` en tu fork con una descripción que incluya:
 - Qué se implementó
 - URL del despliegue (si aplica el extra)
-

Tips

- **Siguiendo la arquitectura existente:** Copiá la estructura de `contexts/tasks/todo/` y adaptá. No reinventes la rueda.
- **DTOs con class-validator:** Mirá `create-todo.dto.ts` como referencia.
- **Tests:** Mirá `todo.service.spec.ts` — el patrón es siempre: mockear el repositorio, llamar al servicio, verificar resultado y que el mock fue llamado.
- **Swagger:** Los decoradores de `@nestjs/swagger` ya están configurados en `main.ts`. Solo necesitás agregarlos en los controllers y DTOs.
- **Relación Category-Todo:** No es obligatorio que cada todo tenga categoría. El campo `categoryId` es opcional.